

How To Train Your Large Language Model

Semester-end Presentation

Yven de Buhr, Joel Dag, Sashreek Nayak Dhaimodkar,
Luke Friedrichs, Jamil Mounzer, Martin Schröder

Supervisor: Prof. Dr. Axel Ngonga



Data Science Group
Paderborn University

July 22, 2025

Contents

- ▶ Challenges
- ▶ Language Dataset
- ▶ Pretraining Approaches
- ▶ Finetuning Approaches
- ▶ Current Problems / Future Work

Introduction

Large Language Models

Rapid Knowledge Access



Productivity & Automation

Flexibility

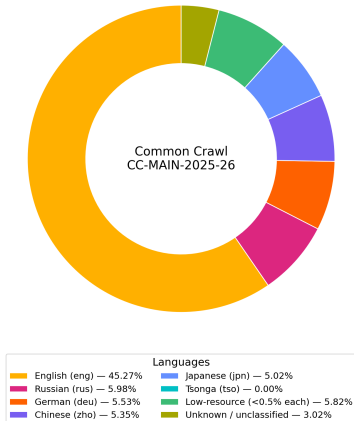


Multilingual Communication

Challenges

Language Imbalance

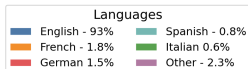
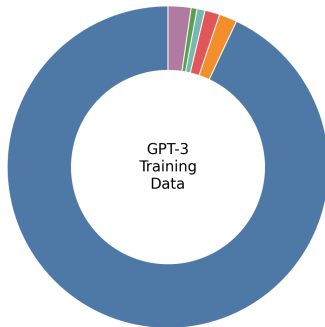
Common Crawl Language Distribution [1]



Challenges

Language Imbalance

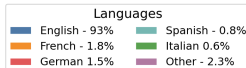
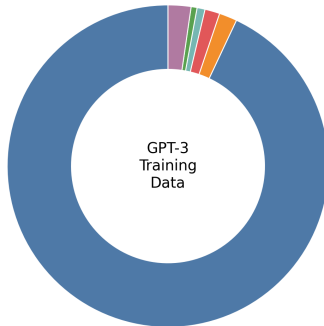
GPT-3 Training Data Language Distribution [2]



Challenges

Language Imbalance

GPT-3 Training Data Language Distribution [2]



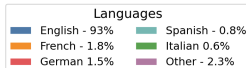
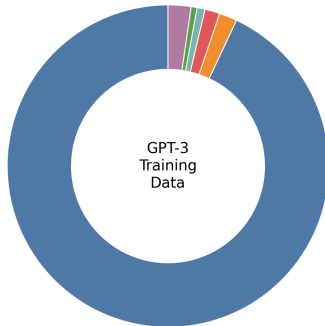
English-centric Bias

- ▶ Models strongly reflect **Anglophone worldview** [3]
- ▶ Marginalization of other cultures / norms [3]

Challenges

Language Imbalance

GPT-3 Training Data Language Distribution [2]



English-centric Bias

- ▶ Models strongly reflect **Anglophone worldview** [3]
- ▶ Marginalization of other cultures / norms [3]

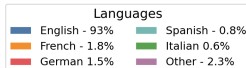
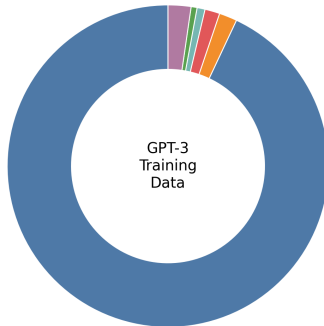
Performance Disparity

- ▶ LLMs show systematically **worse performance** on tasks in low-resource languages [4]

Challenges

Language Imbalance

GPT-3 Training Data Language Distribution [2]



English-centric Bias

- ▶ Models strongly reflect **Anglophone worldview** [3]
- ▶ Marginalization of other cultures / norms [3]

Performance Disparity

- ▶ LLMs show systematically **worse performance** on tasks in low-resource languages [4]

Technological Exclusion

- ▶ 2.191 / 2.485 tracked languages (**88.38%**) have zero known data in official databases [5]

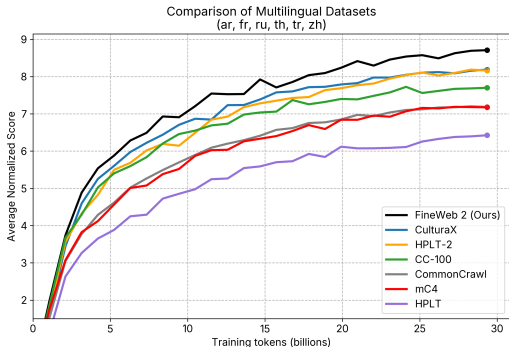
Challenges

Curse of multilinguality

"As dataset sizes increase, adding multilingual data begins to **hurt performance** for **both** low-resource and high-resource languages, likely due to limited model capacity (the "*curse of multilinguality*")." [6]

Task: Train a large and **open-source multilingual** language model and address the challenges posed by the *curse of multilinguality*.

- ▶ Support 500+ languages
- ▶ Ensure computational efficiency
- ▶ Enable multimodal capabilities
- ▶ Maintain linguistic extensibility



Fineweb 2 Dataset [7]

- ▶ 20TB data
- ▶ 1800 languages
- ▶ 5 billion documents

Our subsample

- ▶ 14,000,000 documents in 190 languages

Workflow & Tools

1. **Element Messaging Platform**
(element.cs.uni-paderborn.de)
 - Communication and collaboration
2. **Kanban Board**
(kanboard.cs.uni-paderborn.de)
 - Task visualization, progress tracking, and project workflow management
3. **Sciebo Cloud Storage**
(uni-paderborn.sciebo.de)
 - Secure sharing of files and documents

Technology & Infrastructure

Permanent access

- **Noctua 2** Cluster (128× NVIDIA A100 40 GB) [8]
- **LOLA** GPU VM (2× NVIDIA H100-80GB)

Limited access

- **enexa2** GPU VM (2× *NVIDIA A100 80GB*)
- **morel** GPU VM (4× *NVIDIA H100 96GB*)
- **otus** GPU VM (4× *NVIDIA H100-94GB*)

Section 1 - Pretraining Approaches

Joint Multilingual Pretraining

- ▶ *From-scratch* implementation of the GPT-2 architecture [9]
- ▶ 340M *parameters*
- ▶ Comparable *baseline* reference model

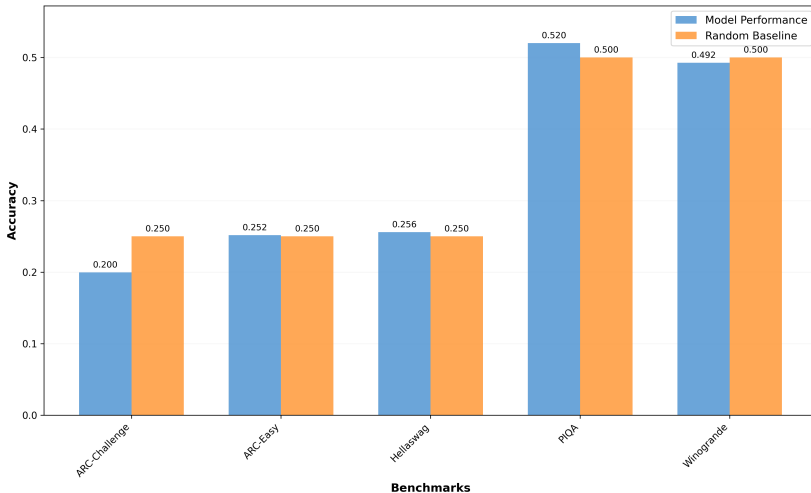
- ▶ *From-scratch* implementation of the GPT-2 architecture [9]
- ▶ 340M *parameters*
- ▶ Comparable *baseline* reference model

Challenges:

- ▶ Very resource intensive
- ▶ *Curse of multilinguality*

Evaluation

Model Performance vs Random Baseline
Across Different Benchmarks



Section 1 - Pretraining Approaches

Mixture of Experts (MoE)

Mixture of Experts (MoE)

- **Key Idea:** Only a few specialized experts are activated per input

Mixture of Experts (MoE)

- ▶ **Key Idea:** Only a few specialized experts are activated per input

Components:

- ▶ **Experts:** Specialized subnetworks trained to handle different data patterns
- ▶ **Router:** Chooses the most relevant experts dynamically

Mixture of Experts (MoE)

- ▶ **Key Idea:** Only a few specialized experts are activated per input

Components:

- ▶ **Experts:** Specialized subnetworks trained to handle different data patterns
- ▶ **Router:** Chooses the most relevant experts dynamically

Benefits:

- ▶ **Efficient** use of compute power
- ▶ **Scalable** model capacity
- ▶ Independent subnetworks specialized in different areas

- ▶ **Key Idea:** Only a few specialized experts are activated per input

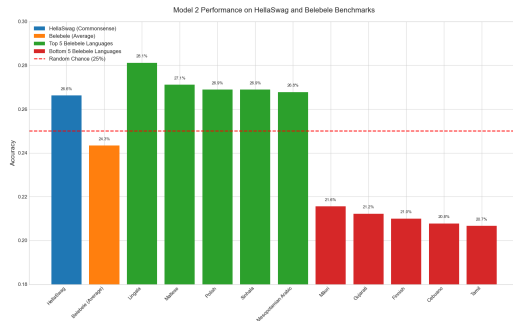
Components:

- ▶ **Experts:** Specialized subnetworks trained to handle different data patterns
- ▶ **Router:** Chooses the most relevant experts dynamically

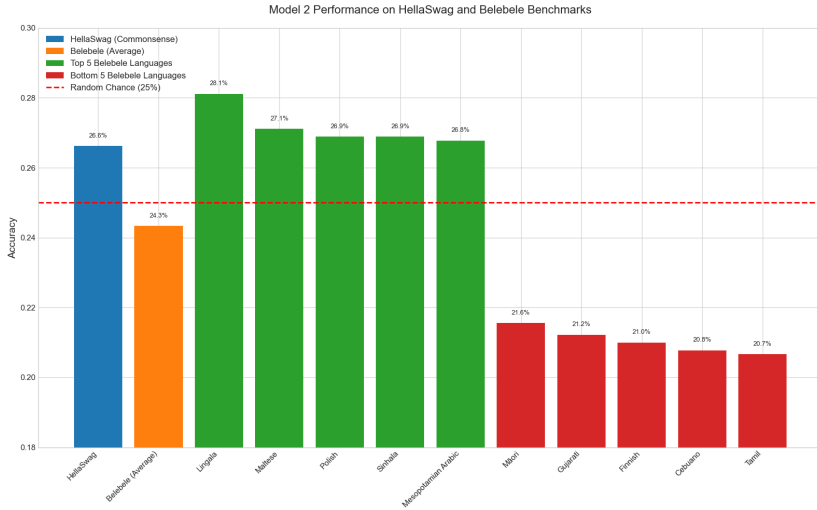
Benefits:

- ▶ **Efficient** use of compute power
 - ▶ **Scalable** model capacity
 - ▶ Independent subnetworks specialized in different areas
-
- ▶ **Multilingual Advantage:** Mitigation of the *curse of multilinguality*

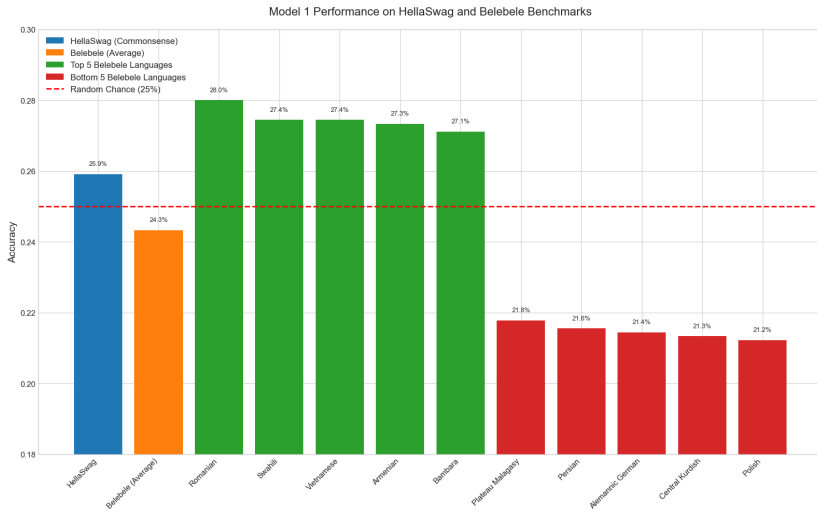
- ▶ MIXTRAL architecture [10]
- ▶ **Own Tokenizer:**
14,850,045,952 *tokens*
- ▶ **Model Size:**
 - ▶ 203,727,360 **total** *parameters*
 - ▶ 47,227,776 **active** *parameters*
- ▶ vocab_size = 131,072
- ▶ 2 experts per token (`n_experts = 7`)
- ▶ 3 training epochs



MoE - Training Run #1

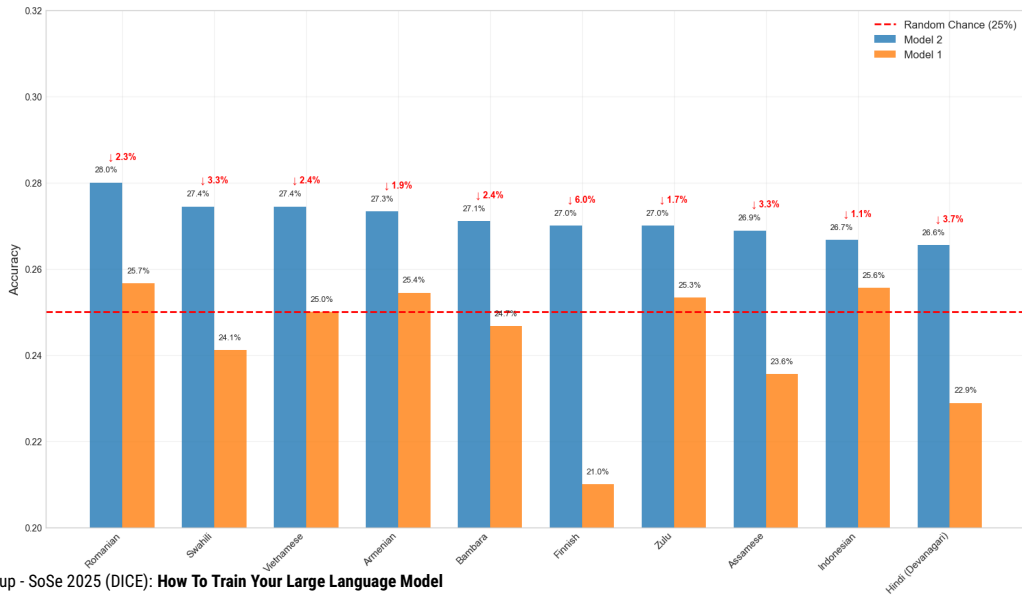


MoE - Training Run #2



Comparison of Run #1 and Run #2

Model 2 vs Model 1 Performance Comparison (Top 10 Languages)



Summary of Run #2 - TFLOPs Profiler

► Headline Metrics

- Overall GPU Utilization: **44.06** TFLOPS (*end-to-end* on **NVIDIA H100 80GB**)
- Training Throughput: **89.2 samples/s** (2 GPUs)

Summary of Run #2 - TFLOPs Profiler

► **Headline Metrics**

- Overall GPU Utilization: **44.06** TFLOPS (*end-to-end* on **NVIDIA H100 80GB**)
- Training Throughput: **89.2 samples/s** (2 GPUs)

► **Latency per Step: 717 ms**

- *Forward Pass* – 24% (175 ms) | 60 TFLOPS
- *Backward Pass* – 61% (438 ms) | 48 TFLOPS (*primary bottleneck*)
- *Optimizer Step* – 15% (104 ms)

Summary of Run #2 - TFLOPs Profiler

► **Headline Metrics**

- Overall GPU Utilization: **44.06** TFLOPS (*end-to-end* on **NVIDIA H100 80GB**)
- Training Throughput: **89.2 samples/s** (2 GPUs)

► **Latency per Step: 717 ms**

- *Forward Pass* – 24% (175 ms) | 60 TFLOPS
- *Backward Pass* – 61% (438 ms) | 48 TFLOPS (*primary bottleneck*)
- *Optimizer Step* – 15% (104 ms)

Critical Bottleneck: MoE Load Imbalance

- Gating network overloads a few experts, leaving others idle.
- **Example** (Layer 0): *Most-used* expert = 52 GFLOPs vs. *least-used* = 5 GFLOPs
- **Result:** Busiest expert waiting, waste of compute cycles

Section 1 - Pretraining Approaches

Follow-up Steps

Megatron-LM

- ▶ GitHub: <https://github.com/NVIDIA/Megatron-LM>
- ▶ Useful for **scaling** models
- ▶ Enables *pipeline-* and *tensor parallelism*

Megatron-LM

- ▶ GitHub: <https://github.com/NVIDIA/Megatron-LM>
- ▶ Useful for **scaling** models
- ▶ Enables *pipeline-* and *tensor parallelism*

Our application

- ▶ $n = 8$ experts, bf16 format, expert_parallelism = 4 (on **NVIDIA A100 40 GB**)
- ▶ **Median:** 214.1 TFLOP/s/GPU (**Min:** 22.7, **Max:** 225.7)
(For reference: Overall GPU Utilization (MoE Run #2): 44.06 TFLOPS)

Section 2 - Finetuning Approaches

Cross-Lingual Finetuning

- ▶ **Idea:** Fine-tune existing model with multilingual data to demonstrate cross-lingual transfer effects, where performance improves for both trained languages and linguistically similar languages.

- ▶ **Idea:** Fine-tune existing model with multilingual data to demonstrate cross-lingual transfer effects, where performance improves for both trained languages and linguistically similar languages.

Challenges:

- ▶ Resource intensive: Fine-tuning requires substantial training relative to model size.
- ▶ *Curse of multilinguality:* Performance drops for unrelated languages not in training data.

- Model: google/gemma-3-4b-pt
- Tokens: 113,191,379

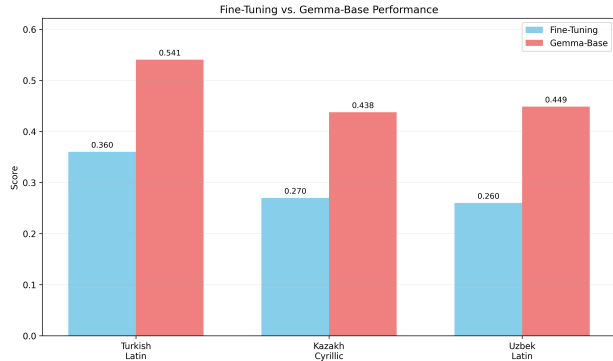


Figure: Fine-tuning on Turkish vs Gemma Base

- Model:
mistralai/Mistral-7B-v0.3
- Tokens: 263,916,337

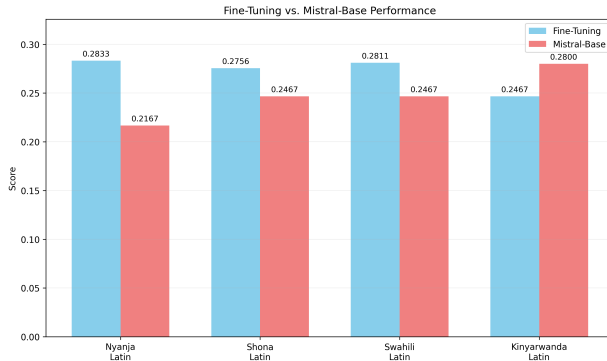


Figure: Fine-tuning on Swahili, Shona, and Nyanja vs Mistral Base

Section 2 - Finetuning Approaches

Adapters

Low-Rank Adaptation of Large Language Models (LoRA)

- Goal: Efficient fine-tuning for large models

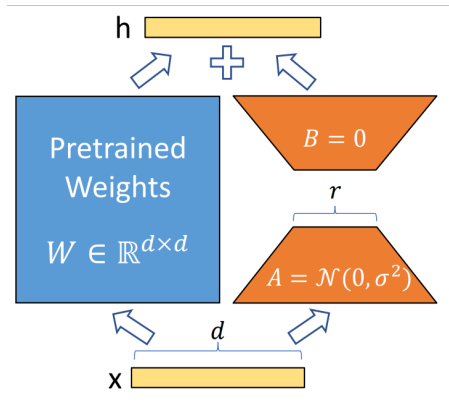


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Low-Rank Adaptation of Large Language Models (LoRA)

- Goal: Efficient fine-tuning for large models
- Freeze original weights $W \in \mathbb{R}^{d \times d}$

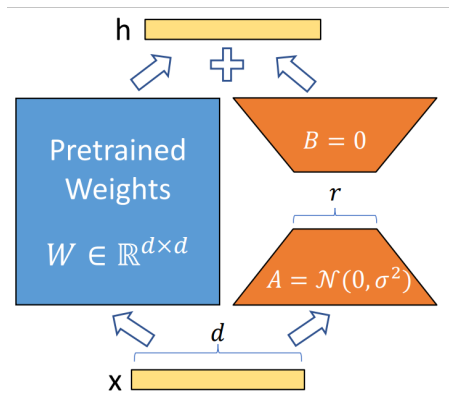


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Low-Rank Adaptation of Large Language Models (LoRA)

- Goal: Efficient fine-tuning for large models
- Freeze original weights $W \in \mathbb{R}^{d \times d}$
- Add trainable low-rank update:
 $\Delta W = BA$

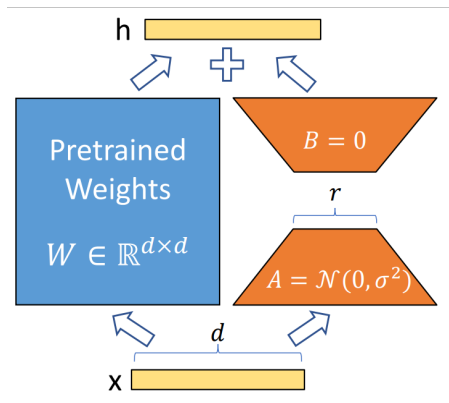


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Low-Rank Adaptation of Large Language Models (LoRA)

- Goal: Efficient fine-tuning for large models
- Freeze original weights $W \in \mathbb{R}^{d \times d}$
- Add trainable low-rank update:
 $\Delta W = BA$
 - $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - $r \ll \min(d, k)$

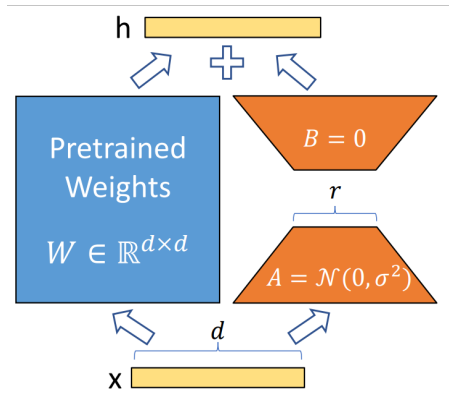


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Low-Rank Adaptation of Large Language Models (LoRA)

- ▶ Goal: Efficient fine-tuning for large models
- ▶ Freeze original weights $W \in \mathbb{R}^{d \times d}$
- ▶ Add trainable low-rank update: $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$
- ▶ Modified projection: $W'x = Wx + BAx$

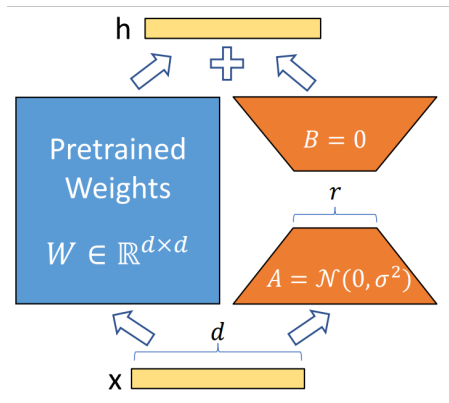


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Low-Rank Adaptation of Large Language Models (LoRA)

- ▶ Goal: Efficient fine-tuning for large models
- ▶ Freeze original weights $W \in \mathbb{R}^{d \times d}$
- ▶ Add trainable low-rank update:
 $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$
- ▶ Modified projection: $W'x = Wx + BAx$
- ▶ Only train A, B with significantly fewer parameters

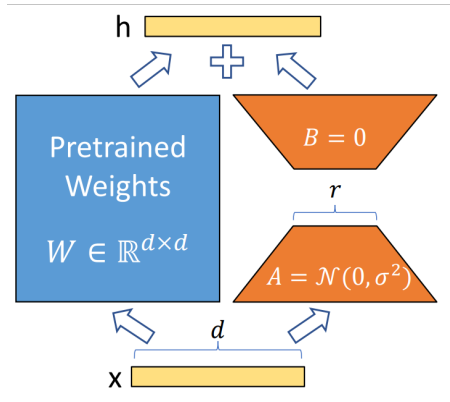


Figure: LoRA: Injecting low-rank updates into frozen pretrained weights [11]

Language Groups

- ▶ **Niger-Congo (Latin):** swl, nso, zul, tsn, sot, yor, ibo, ewe, fon
- ▶ **Arabic (Arabic):** ars, apc, arb, acm, arz
- ▶ **Cyrillic:** kaz, khk
- ▶ **Bengali (Bengali):** asm, ben
- ▶ **Georgian (Georgian):** kat, xmf

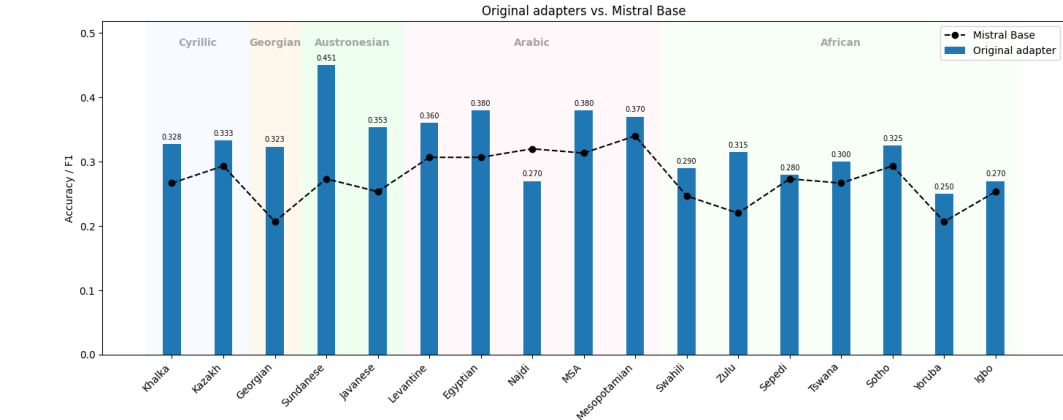
Model Setup

- ▶ Base: Mistral-7B
- ▶ Fine-tuned via: LoRA/QLoRA adapters
- ▶ Adapter size: **7.1M** parameters

Training Data (Tokens)

- ▶ Niger-Congo: **825M**
- ▶ Arabic: **602M**
- ▶ Bengali: **905M**
- ▶ Cyrillic: **205M**
- ▶ Georgian: **104M**

Evaluation and results



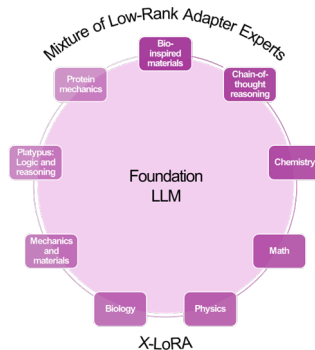
Direct Comparison: QLoRA Adapter vs Full Fine-Tuning vs Base

Table: Finetuning Mistral 7B on Swahili, Shona, and Nyanja reading comprehension (Belebele benchmark)

Model	Nyanja (nya)	Shona (sna)	Swahili (swh)	Total Score
Full Fine-Tuning	0.2833	0.2756	0.2811	0.81
QLoRA Adapter (Step 2505)	0.2944	0.3422	0.3322	0.9689
Mistral 7B Base	0.2467	0.2467	0.2467	0.7400

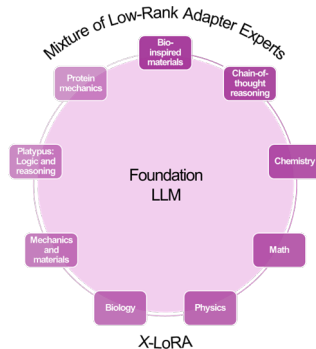
- ▶ Belebele reading comprehension tasks in low-resource languages.
- ▶ Improvement from near-random baseline to effective comprehension through fine-tuning.
- ▶ QLoRA captures much of the gain from full fine-tuning with far less compute.

- **Goal:** Use a mixture of LoRA adapters dynamically during inference



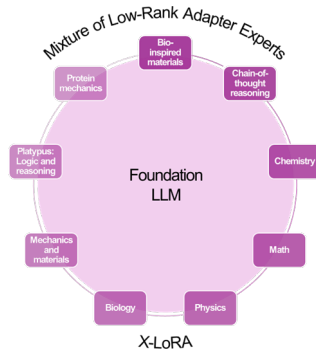
Mixture of Low-Rank Adapter Experts (XLoRA)

- **Goal:** Use a mixture of LoRA adapters dynamically during inference
- Freeze pretrained weights $W_0 \in \mathbb{R}^{d \times k}$



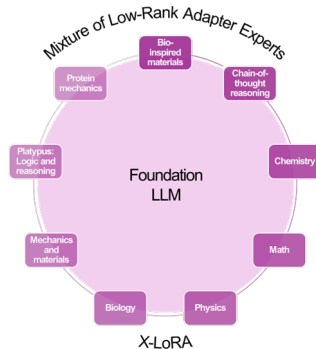
Mixture of Low-Rank Adapter Experts (XLoRA)

- **Goal:** Use a mixture of LoRA adapters dynamically during inference
- Freeze pretrained weights $W_0 \in \mathbb{R}^{d \times k}$
- Add multiple low-rank adapters: $\Delta W_i = B_i A_i$, for $i = 1, \dots, N$

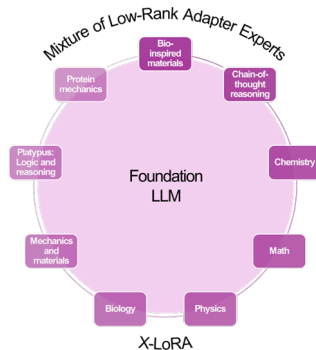


Mixture of Low-Rank Adapter Experts (XLoRA)

- **Goal:** Use a mixture of LoRA adapters dynamically during inference
- Freeze pretrained weights $W_0 \in \mathbb{R}^{d \times k}$
- Add multiple low-rank adapters: $\Delta W_i = B_i A_i$, for $i = 1, \dots, N$
 - $B_i \in \mathbb{R}^{d \times r}$, $A_i \in \mathbb{R}^{r \times k}$
 - $r \ll \min(d, k)$

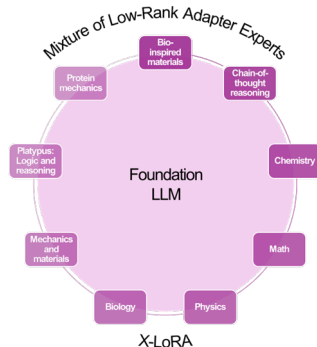


- **Goal:** Use a mixture of LoRA adapters dynamically during inference
- Freeze pretrained weights $W_0 \in \mathbb{R}^{d \times k}$
- Add multiple low-rank adapters: $\Delta W_i = B_i A_i$, for $i = 1, \dots, N$
 - $B_i \in \mathbb{R}^{d \times r}$, $A_i \in \mathbb{R}^{r \times k}$
 - $r \ll \min(d, k)$
- Forward pass becomes:
$$h = W_0 x + \Delta W_0 x = W_0 x + B A x$$



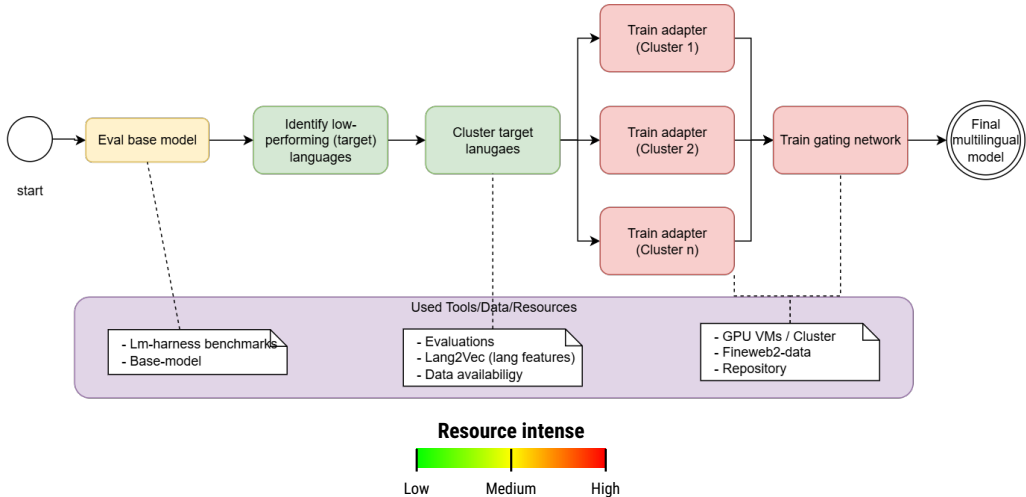
Mixture of Low-Rank Adapter Experts (XLoRA)

- ▶ **Goal:** Use a mixture of LoRA adapters dynamically during inference
- ▶ Freeze pretrained weights $W_0 \in \mathbb{R}^{d \times k}$
- ▶ Add multiple low-rank adapters: $\Delta W_i = B_i A_i$, for $i = 1, \dots, N$
 - ▶ $B_i \in \mathbb{R}^{d \times r}$, $A_i \in \mathbb{R}^{r \times k}$
 - ▶ $r \ll \min(d, k)$
- ▶ Forward pass becomes:
$$h = W_0 x + \Delta W_0 x = W_0 x + B A x$$
- ▶ Each adapter's weight is scaled per token and layer: $\alpha_i^* = \alpha_i \cdot \lambda_i$
where λ_i is predicted by a learned scaling head based on hidden states.

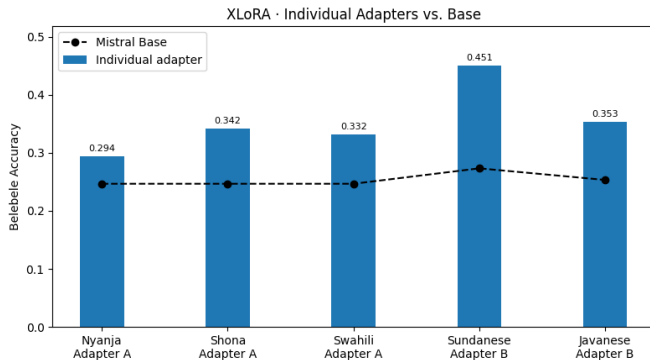


Mixture of Low-Rank Adapter Experts

Workflow of multilingulizing a base model

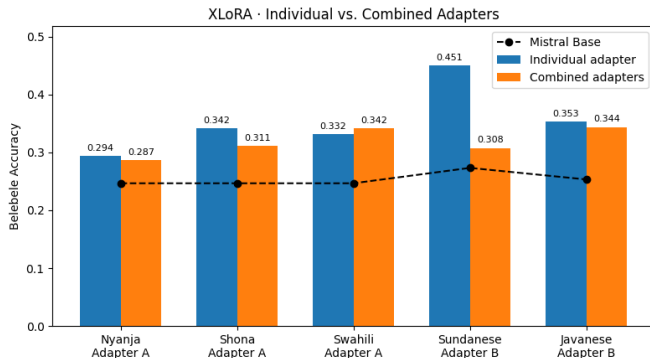


XLoRA Results: Individual vs. Combined Adapters



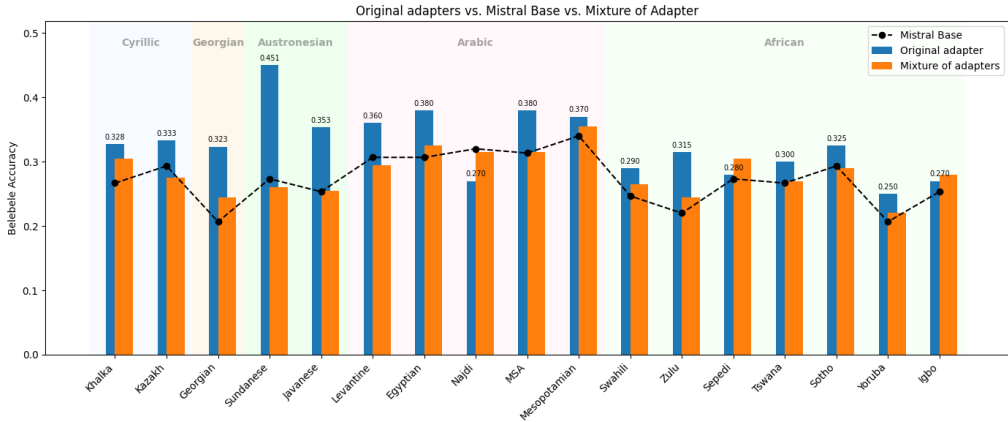
► Belebele reading comprehension benchmark

XLoRA Results: Individual vs. Combined Adapters



- Belebele reading comprehension benchmark
- Individual adapters perform stronger
- Gating decreases performance

XLoRA: Current Problems



- ▶ When scaling up the amount of languages, the Gating degrades most of the adapter improvements
- ▶ Work in Progress: More fine scaled gating implementation

Section 4

Current Problems / Future Work

- ▶ **Tokenizer strategy:** single big tokenizer/ multiple individual ones?
- ▶ Only employ gating if *high confidence*, otherwise use base model.
- ▶ Combination of language adapters during inference?
- ▶ Figure out **optimal language clusters** for adapters.
- ▶ Counteract/ Mitigate **MoE Load Imbalance**.

Four approaches to explore

- ▶ Joint Multilingual Pretraining
- ▶ Mixture of Experts
- ▶ Cross-Lingual Transfer Learning
- ▶ Language Adapters

Three main challenges

- ▶ Curse of multilinguality
- ▶ Training efficiency
- ▶ Low-resource languages

Future Work

- ▶ **Tokenizer strategy**: single big tokenizer/ multiple individual ones?
- ▶ Only employ gating if *high confidence*, otherwise use base model.
- ▶ Combination of language adapters during inference?
- ▶ Figure out **optimal language clusters** for adapters.
- ▶ Counteract/ Mitigate **MoE Load Imbalance**.

Thank you!

Questions?

HTYLLM Project Group (SoSe 2025) at Paderborn University

Code: github.com/dice-group/HTYLLM-PG

- [1] "Statistics of Common Crawl Monthly Archives by commoncrawl – commoncrawl.github.io."
<https://commoncrawl.github.io/cc-crawl-statistics/plots/languages>.
[Accessed 20-07-2025].
- [2] R. L. Johnson, G. Pistilli, N. Menéndez-González, L. D. D. Duran, E. Panai, J. Kalpokiene, and D. J. Bertulfo, "The ghost in the machine has an american accent: value conflict in gpt-3," *arXiv preprint arXiv:2203.07785*, 2022.
- [3] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the dangers of stochastic parrots: Can language models be too big?," in *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pp. 610–623, 2021.
- [4] D. Blasi, A. Anastasopoulos, and G. Neubig, "Systematic inequalities in language technology performance across the world's languages," *arXiv preprint arXiv:2110.06733*, 2021.
- [5] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the nlp world," *arXiv preprint arXiv:2004.09095*, 2020.
- [6] T. A. Chang, C. Arnett, Z. Tu, and B. K. Bergen, "When is multilinguality a curse? language modeling for 250 high- and low-resource languages," 2023.
- [7] "HuggingFaceFW/fineweb-2 · Datasets at Hugging Face – huggingface.co."
<https://huggingface.co/datasets/HuggingFaceFW/fineweb-2>.
[Accessed 21-07-2025].

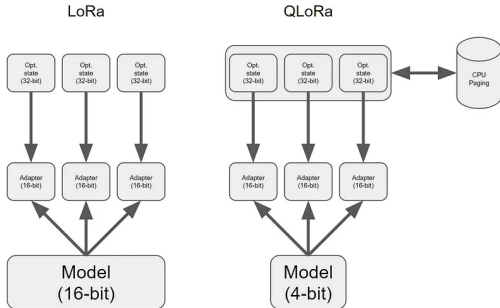
- [8] C. Bauer, T. Kenter, M. Lass, L. Mazur, M. Meyer, H. Nitsche, H. Riebler, R. Schade, M. Schwarz, N. Winnwa, *et al.*, “Noctua 2 supercomputer,” *Journal of large-scale research facilities JLSRF*, vol. 9, no. 1, 2024.
- [9] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.*, “Language models are unsupervised multitask learners,” *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [10] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. I. Casas, E. B. Hanna, F. Bressand, *et al.*, “Mixtral of experts,” *arXiv preprint arXiv:2401.04088*, 2024.
- [11] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, *et al.*, “Lora: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [12] N. Srivastava, D. Kuchelev, T. M. Ngoli, K. Shetty, M. Röder, H. Zahera, D. Moussallem, and A.-C. N. Ngomo, “Lola – an open-source massively multilingual large language model,” 2025.
- [13] A. Chronopoulou, D. Stojanovski, and A. Fraser, “Language-family adapters for low-resource multilingual neural machine translation,” *arXiv preprint arXiv:2209.15236*, 2022.
- [14] J. Pfeiffer, I. Vulić, I. Gurevych, and S. Ruder, “Mad-x: An adapter-based framework for multi-task cross-lingual transfer,” *arXiv preprint arXiv:2005.00052*, 2020.
- [15] HuggingFace, “Huggingfacefw/fineweb.”

[16] HuggingFace, "Huggingfacefw/fineweb-2."

Section 3

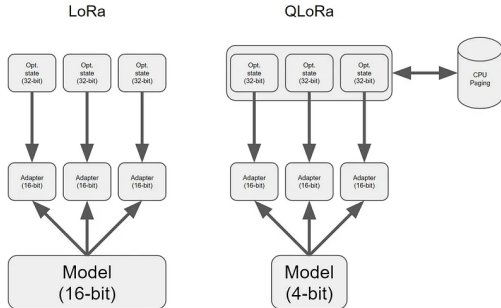
Backup Slides

QLoRA: Quantized LoRA



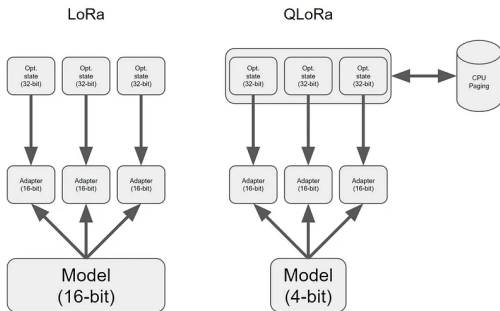
- TODO: remove and only include if we need more content
- Memory-efficient fine-tuning for large models

QLoRA: Quantized LoRA



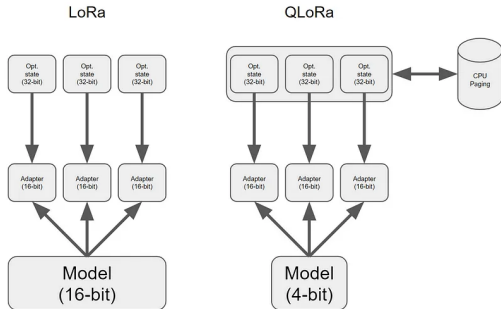
- TODO: remove and only include if we need more content
- Memory-efficient fine-tuning for large models
- **Quantize** base model to 4-bit

QLoRA: Quantized LoRA

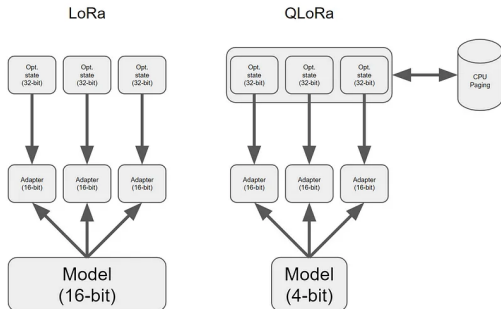


- TODO: remove and only include if we need more content
- Memory-efficient fine-tuning for large models
- **Quantize** base model to 4-bit
- **Freeze** quantized weights during training

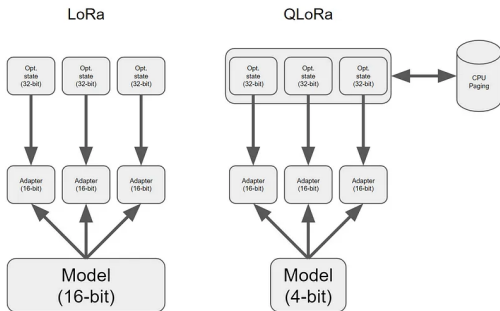
QLoRA: Quantized LoRA



- TODO: remove and only include if we need more content
- Memory-efficient fine-tuning for large models
- **Quantize** base model to 4-bit
- **Freeze** quantized weights during training
- **Insert LoRA adapters:** $\Delta W = BA$

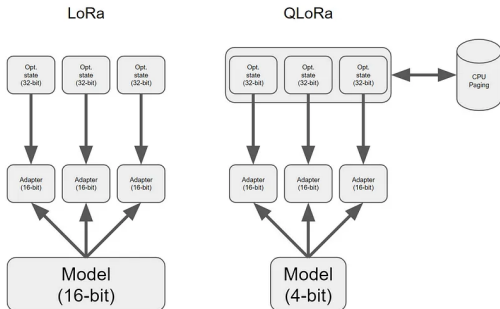


- ▶ TODO: remove and only include if we need more content
- ▶ Memory-efficient fine-tuning for large models
- ▶ **Quantize** base model to 4-bit
- ▶ **Freeze** quantized weights during training
- ▶ **Insert LoRA adapters:** $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$



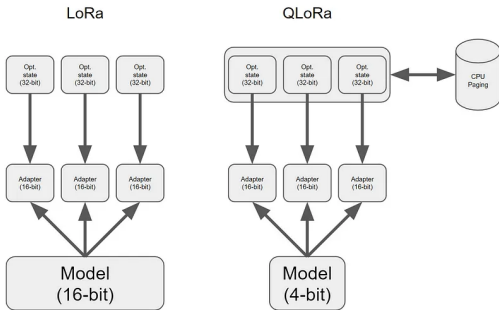
- ▶ TODO: remove and only include if we need more content
- ▶ Memory-efficient fine-tuning for large models
- ▶ **Quantize** base model to 4-bit
- ▶ **Freeze** quantized weights during training
- ▶ **Insert LoRA adapters:** $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}, \quad B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$
- ▶ Forward pass: $W'x = Wx + BAx$

QLoRA: Quantized LoRA



- ▶ TODO: remove and only include if we need more content
- ▶ Memory-efficient fine-tuning for large models
- ▶ **Quantize** base model to 4-bit
- ▶ **Freeze** quantized weights during training
- ▶ **Insert LoRA adapters:** $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$
- ▶ Forward pass: $W'x = Wx + BAx$
- ▶ Only train A, B ; rest is 4-bit frozen

QLoRA: Quantized LoRA



- ▶ TODO: remove and only include if we need more content
- ▶ Memory-efficient fine-tuning for large models
- ▶ **Quantize** base model to 4-bit
- ▶ **Freeze** quantized weights during training
- ▶ **Insert LoRA adapters:** $\Delta W = BA$
 - ▶ $A \in \mathbb{R}^{r \times k}$, $B \in \mathbb{R}^{d \times r}$
 - ▶ $r \ll \min(d, k)$
- ▶ Forward pass: $W'x = Wx + BAx$
- ▶ Only train A, B ; rest is 4-bit frozen
- ▶ Enables fine-tuning models more compute efficiently